

PRE-PROCESSING OF GENERATED DATASET FROM DWSIM SOFTWARE

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: df = pd.read_csv('Dataset.csv')
df
```

Out[2]:

	DCOL-1 - Condenser_Specification_Value	DCOL-1 - Reboiler_Specification_Value	Distillate - Molar Fraction (Overall Liquid) / Benzene ()	Bottoms - Molar Fraction (Overall Liquid) / Toluene ()	E1 - Energy Flow (kW)	E2 - Energy Flow (kW)
0	1.2	1.0	0.546505	0.999809	770.004	884.623
1	1.2	1.5	0.573153	0.999753	732.024	847.190
2	1.2	2.0	0.602525	0.999674	693.931	809.684
3	1.2	2.5	0.635054	0.999558	655.708	772.093
4	1.2	3.0	0.671267	0.999380	617.336	734.404
...	...	...	...	...	...	...
115	5.0	1.0	0.546518	0.999946	2100.010	2214.630
116	5.0	1.5	0.573180	0.999938	1996.420	2111.590
117	5.0	2.0	0.602577	0.999929	1892.530	2008.280
118	5.0	2.5	0.635151	0.999917	1788.270	1904.660
119	5.0	3.0	0.671446	0.999901	1683.600	1800.680

120 rows × 6 columns

```
In [3]: print("---RAW DATASET HEAD---")
print(df.head())
```

---RAW DATASET HEAD---

	DCOL-1 - Condenser_Specification_Value \
0	1.2
1	1.2
2	1.2
3	1.2
4	1.2

	DCOL-1 - Reboiler_Specification_Value \
0	1.0
1	1.5
2	2.0
3	2.5
4	3.0

	Distillate - Molar Fraction (Overall Liquid) / Benzene () \
0	0.546505
1	0.573153
2	0.602525
3	0.635054
4	0.671267

	Bottoms - Molar Fraction (Overall Liquid) / Toluene () \
0	0.999809
1	0.999753
2	0.999674
3	0.999558
4	0.999380

	E1 - Energy Flow (kW)	E2 - Energy Flow (kW)
0	770.004	884.623
1	732.024	847.190
2	693.931	809.684
3	655.708	772.093
4	617.336	734.404

```
In [4]: df.columns = ['Reflux_Ratio', 'Reboiler_Spec', 'x_D', 'x_B', 'Q_C', 'Q_R']
df
```

Out[4]:

	Reflux_Ratio	Reboiler_Spec	x_D	x_B	Q_C	Q_R
0	1.2	1.0	0.546505	0.999809	770.004	884.623
1	1.2	1.5	0.573153	0.999753	732.024	847.190
2	1.2	2.0	0.602525	0.999674	693.931	809.684
3	1.2	2.5	0.635054	0.999558	655.708	772.093
4	1.2	3.0	0.671267	0.999380	617.336	734.404
...	...	...	...	...	...	...
115	5.0	1.0	0.546518	0.999946	2100.010	2214.630
116	5.0	1.5	0.573180	0.999938	1996.420	2111.590
117	5.0	2.0	0.602577	0.999929	1892.530	2008.280
118	5.0	2.5	0.635151	0.999917	1788.270	1904.660
119	5.0	3.0	0.671446	0.999901	1683.600	1800.680

120 rows × 6 columns

```
In [5]: df_clean = df.dropna().copy()
df_clean = df_clean[
    (df_clean['x_D'] >= 0) & (df_clean['x_D'] <= 1) &
    (df_clean['x_B'] >= 0) & (df_clean['x_B'] <= 1) &
    (df_clean['Q_C'] > 0) &
    (df_clean['Q_R'] > 0)
]

print("\n--- CLEANED DATASET SHAPE ---")
print(f"Original rows: {len(df)}, Cleaned rows: {len(df_clean)}")
```

```
--- CLEANED DATASET SHAPE ---
Original rows: 120, Cleaned rows: 120
```

In [6]: df.shape

Out[6]: (120, 6)

In [7]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Reflux_Ratio  120 non-null    float64
1   Reboiler_Spec 120 non-null    float64
2   x_D           120 non-null    float64
3   x_B           120 non-null    float64
4   Q_C           120 non-null    float64
5   Q_R           120 non-null    float64
dtypes: float64(6)
memory usage: 5.8 KB
```

In [8]: `df.describe()`

Out[8]:

	Reflux_Ratio	Reboiler_Spec	x_D	x_B	Q_C	Q_R
count	120.000000	120.000000	120.000000	120.000000	120.000000	120.000000
mean	3.100000	2.000000	0.605754	0.999844	1292.983000	1408.783650
std	1.148461	0.710072	0.044351	0.000101	377.056235	376.824865
min	1.200000	1.000000	0.546505	0.999380	617.336000	734.404000
25%	2.149998	1.500000	0.573168	0.999811	979.779750	1096.522500
50%	3.100000	2.000000	0.602567	0.999876	1286.100000	1400.995000
75%	4.050002	2.500000	0.635144	0.999911	1590.985000	1707.757500
max	5.000000	3.000000	0.671446	0.999946	2100.010000	2214.630000

MACHINE LEARNING MODELS TRAINING ON CLEAN GENERATED DATASET TO IDENTIFY THE BEST SURROGATE MODEL

```
In [9]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [10]: df.columns = df.columns.str.strip()
```

```
In [11]: X = df[['Reflux_Ratio', 'Reboiler_Spec']]
Y = df[['x_D', 'x_B', 'Q_C', 'Q_R']]
```

```
In [12]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

```
In [13]: scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)
```

```
In [14]: scaler_Y = StandardScaler()
Y_train_scaled = scaler_Y.fit_transform(Y_train)
```

```
In [15]: poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
poly_model.fit(X_train, Y_train)
Y_pred_poly = poly_model.predict(X_test)
```

```
In [16]: rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, Y_train)
Y_pred_rf = rf_model.predict(X_test)
```

```
In [17]: ann_model = MLPRegressor(hidden_layer_sizes=(64, 64), activation='relu', max_iter=2000, random_state=42)
ann_model.fit(X_train_scaled, Y_train_scaled)
Y_pred_ann = scaler_Y.inverse_transform(ann_model.predict(X_test_scaled))
```

EVALUATION OF MAE, RMSE AND R2

```
In [18]: def evaluate(Y_true, Y_pred, model_name):
    records = []
    target_names = ['x_D', 'x_B', 'Q_C', 'Q_R']
    for i, col in enumerate(target_names):
        mae = mean_absolute_error(Y_true.iloc[:, i], Y_pred[:, i])
        rmse = np.sqrt(mean_squared_error(Y_true.iloc[:, i], Y_pred[:, i]))
        r2 = r2_score(Y_true.iloc[:, i], Y_pred[:, i])
        records.append({'Model': model_name, 'Target': col, 'MAE': mae, 'RMSE': rmse, 'R2': r2})
    return pd.DataFrame(records)
```

```
In [19]: eval_df = pd.concat([
    evaluate(Y_test, Y_pred_poly, 'Polynomial Regression (Deg 2)'),
    evaluate(Y_test, Y_pred_rf, 'Random Forest'),
    evaluate(Y_test, Y_pred_ann, 'ANN')
], ignore_index=True)

print(eval_df.to_string())
```

	Model	Target	MAE	RMSE	R2
0	Polynomial Regression (Deg 2)	x_D	0.000125	0.000138	0.999992
1	Polynomial Regression (Deg 2)	x_B	0.000016	0.000032	0.915369
2	Polynomial Regression (Deg 2)	Q_C	0.050164	0.066227	1.000000
3	Polynomial Regression (Deg 2)	Q_R	0.049885	0.065327	1.000000
4	Random Forest	x_D	0.010312	0.012131	0.938990
5	Random Forest	x_B	0.000018	0.000035	0.899173
6	Random Forest	Q_C	39.324003	45.800629	0.978482
7	Random Forest	Q_R	39.149171	45.590681	0.978680
8	ANN	x_D	0.001016	0.001276	0.999325
9	ANN	x_B	0.000014	0.000037	0.886242
10	ANN	Q_C	18.651241	23.957910	0.994112
11	ANN	Q_R	16.490979	19.442506	0.996123

```
In [20]: print("="*80)
    print("                MODEL EVALUATION SUMMARY RESULTS                ")
    print("="*80)
    print(eval_df.to_string(index=False))
```

=====					
MODEL EVALUATION SUMMARY RESULTS					
=====					
	Model	Target	MAE	RMSE	R2
Polynomial Regression (Deg 2)	x_D	0.000125	0.000138	0.999992	
Polynomial Regression (Deg 2)	x_B	0.000016	0.000032	0.915369	
Polynomial Regression (Deg 2)	Q_C	0.050164	0.066227	1.000000	
Polynomial Regression (Deg 2)	Q_R	0.049885	0.065327	1.000000	
Random Forest	x_D	0.010312	0.012131	0.938990	
Random Forest	x_B	0.000018	0.000035	0.899173	
Random Forest	Q_C	39.324003	45.800629	0.978482	
Random Forest	Q_R	39.149171	45.590681	0.978680	
ANN	x_D	0.001016	0.001276	0.999325	
ANN	x_B	0.000014	0.000037	0.886242	
ANN	Q_C	18.651241	23.957910	0.994112	
ANN	Q_R	16.490979	19.442506	0.996123	

```
In [21]: print("\n" + "="*80)
print("                                OVERALL AVERAGE METRICS                                ")
print("="*80)
print(eval_df.groupby('Model')[['MAE', 'RMSE', 'R2']].mean())
```

=====					
OVERALL AVERAGE METRICS					
=====					
		MAE	RMSE	R2	
Model					
ANN		8.785812	10.850432	0.968950	
Polynomial Regression (Deg 2)		0.025048	0.032931	0.978840	
Random Forest		19.620876	22.850869	0.948831	

```
In [22]: print("\n" + "="*80)
print("                                PHYSICAL CONSISTENCY CHECK                                ")
print("="*80)
```

=====					
PHYSICAL CONSISTENCY CHECK					
=====					

```
In [23]: models_predictions = {
        'Polynomial Regression (Deg 2)': Y_pred_poly,
```

```
'Random Forest': Y_pred_rf,
'ANN': Y_pred_ann
}
```

VALIDATION CHECK

```
In [24]: for name, pred in models_predictions.items():
min_xD, max_xD = pred[:, 0].min(), pred[:, 0].max()
min_xB, max_xB = pred[:, 1].min(), pred[:, 1].max()
min_QC = pred[:, 2].min()
min_QR = pred[:, 3].min()

valid_bounds = (0 <= min_xD <= 1) and (0 <= max_xD <= 1) and (0 <= min_xB <= 1) and (0 <= max_xB <= 1)
valid_duties = (min_QC > 0) and (min_QR > 0)

print(f"\n[{name}]\n")
print(f" - x_D Range: [{min_xD:.4f}, {max_xD:.4f}] -> Valid [0, 1]: {valid_bounds}")
print(f" - x_B Range: [{min_xB:.4f}, {max_xB:.4f}] -> Valid [0, 1]: {valid_bounds}")
print(f" - Duties > 0 kW: QC_min = {min_QC:.2f} kW, QR_min = {min_QR:.2f} kW -> Valid: {valid_duties}")
```

[Polynomial Regression (Deg 2)]

```
- x_D Range: [0.5466, 0.6713] -> Valid [0, 1]: True
- x_B Range: [0.9995, 0.9999] -> Valid [0, 1]: True
- Duties > 0 kW: QC_min = 617.20 kW, QR_min = 734.27 kW -> Valid: True
```

[Random Forest]

```
- x_D Range: [0.5611, 0.6658] -> Valid [0, 1]: True
- x_B Range: [0.9995, 0.9999] -> Valid [0, 1]: True
- Duties > 0 kW: QC_min = 667.22 kW, QR_min = 783.87 kW -> Valid: True
```

[ANN]

```
- x_D Range: [0.5439, 0.6722] -> Valid [0, 1]: True
- x_B Range: [0.9996, 0.9999] -> Valid [0, 1]: True
- Duties > 0 kW: QC_min = 659.62 kW, QR_min = 730.36 kW -> Valid: True
```

DATA VISUALIZATION

PARITY PLOTS

```
In [25]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

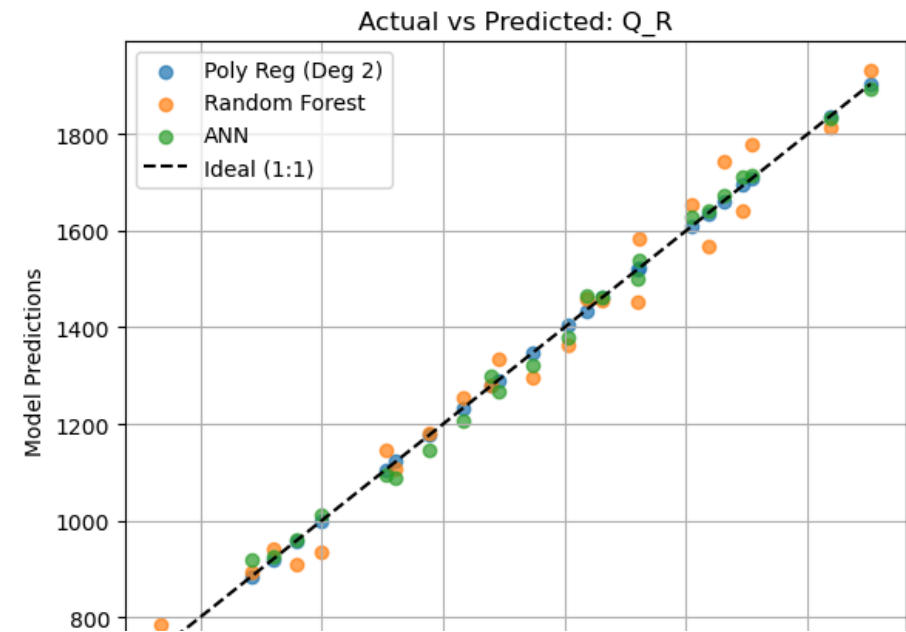
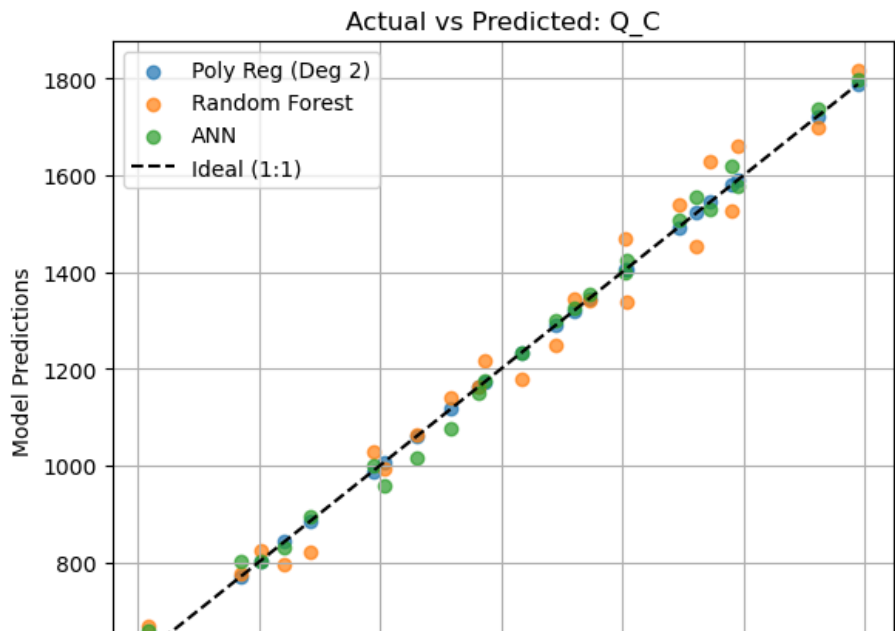
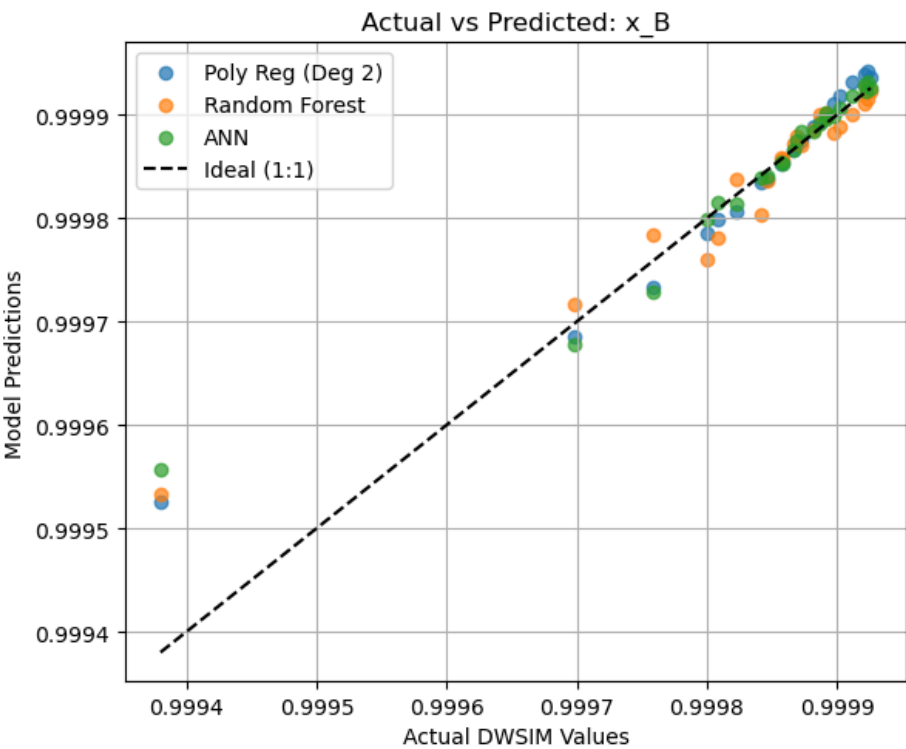
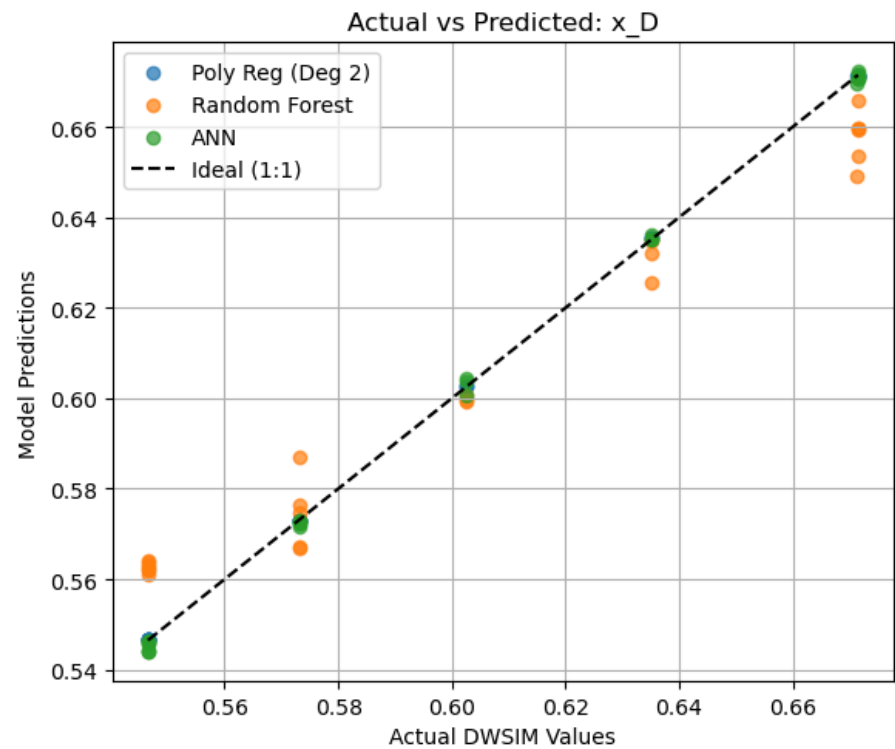
target_names = ['x_D', 'x_B', 'Q_C', 'Q_R']

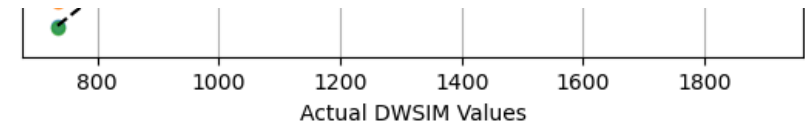
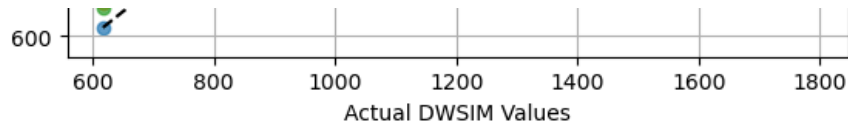
for i, col_name in enumerate(target_names):
    axes[i].scatter(Y_test.iloc[:, i], Y_pred_poly[:, i], label='Poly Reg (Deg 2)', alpha=0.7)
    axes[i].scatter(Y_test.iloc[:, i], Y_pred_rf[:, i], label='Random Forest', alpha=0.7)
    axes[i].scatter(Y_test.iloc[:, i], Y_pred_ann[:, i], label='ANN', alpha=0.7)

    # 1:1 Reference Line
    min_val = Y_test.iloc[:, i].min()
    max_val = Y_test.iloc[:, i].max()
    axes[i].plot([min_val, max_val], [min_val, max_val], 'k--', label='Ideal (1:1)')

    axes[i].set_title(f'Actual vs Predicted: {col_name}')
    axes[i].set_xlabel('Actual DWSIM Values')
    axes[i].set_ylabel('Model Predictions')
    axes[i].legend()
    axes[i].grid(True)

plt.tight_layout()
plt.show()
```





HEATMAPS

```
In [33]: # Prepare Pivot Tables for Heatmaps
pivot_xD = df.pivot(index='Reflux_Ratio', columns='Reboiler_Spec', values='x_D')
pivot_QR = df.pivot(index='Reflux_Ratio', columns='Reboiler_Spec', values='Q_R')

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

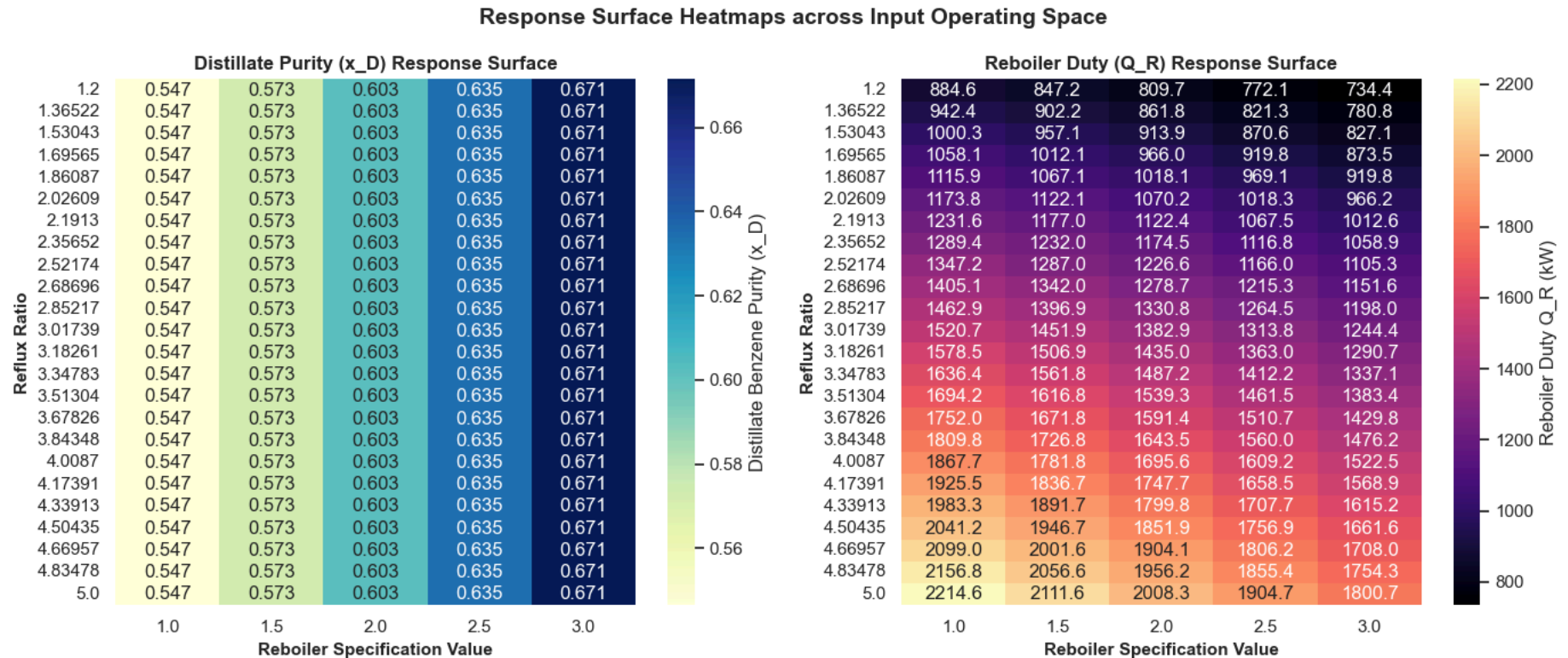
# Subplot 1: Distillate Purity Heatmap
sns.heatmap(
    pivot_xD,
    annot=True,
    fmt=".3f",
    cmap="YlGnBu",
    ax=axes[0],
    cbar_kws={'label': 'Distillate Benzene Purity (x_D)'}
)
axes[0].set_title('Distillate Purity (x_D) Response Surface', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Reboiler Specification Value', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Reflux Ratio', fontsize=11, fontweight='bold')

# Subplot 2: Reboiler Duty Heatmap
sns.heatmap(
    pivot_QR,
    annot=True,
    fmt=".1f",
    cmap="magma",
    ax=axes[1],
    cbar_kws={'label': 'Reboiler Duty Q_R (kW)'}
)
axes[1].set_title('Reboiler Duty (Q_R) Response Surface', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Reboiler Specification Value', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Reflux Ratio', fontsize=11, fontweight='bold')

plt.suptitle('Response Surface Heatmaps across Input Operating Space', fontsize=14, fontweight='bold')
```

```
plt.tight_layout()

plt.savefig('Response_Surface_Heatmaps.png', dpi=300)
plt.show()
```



PHYSICAL SENSITIVITY TREND LINES THROUGH GRAPH VISUALIZATION

```
In [34]: fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Subplot 1: Effect of Reflux Ratio on Energy Duties (at Reboiler Spec = 1.0)
sample_reboiler_spec = 1.0
df_sub = df[df['Reboiler_Spec'] == sample_reboiler_spec].sort_values('Reflux_Ratio')

axes[0].plot(df_sub['Reflux_Ratio'], df_sub['Q_C'], 'o-', label='Condenser Duty (Q_C)', color='#1f77b4', linewidth=2, markersize=10)
axes[0].plot(df_sub['Reflux_Ratio'], df_sub['Q_R'], 's-', label='Reboiler Duty (Q_R)', color='#d62728', linewidth=2, markersize=10)
axes[0].set_title(f'Energy Duties vs. Reflux Ratio\n(At Reboiler Spec = {sample_reboiler_spec})', fontsize=12, fontweight='bold')
```

```

axes[0].set_xlabel('Reflux Ratio', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Energy Duty (kW)', fontsize=11, fontweight='bold')
axes[0].legend(loc='upper left')
axes[0].grid(True, linestyle='--', alpha=0.6)

# Subplot 2: Effect of Reboiler Spec on Distillate Purity (x_D)
df_sub_reboiler = df.groupby('Reboiler_Spec')['x_D'].mean().reset_index()

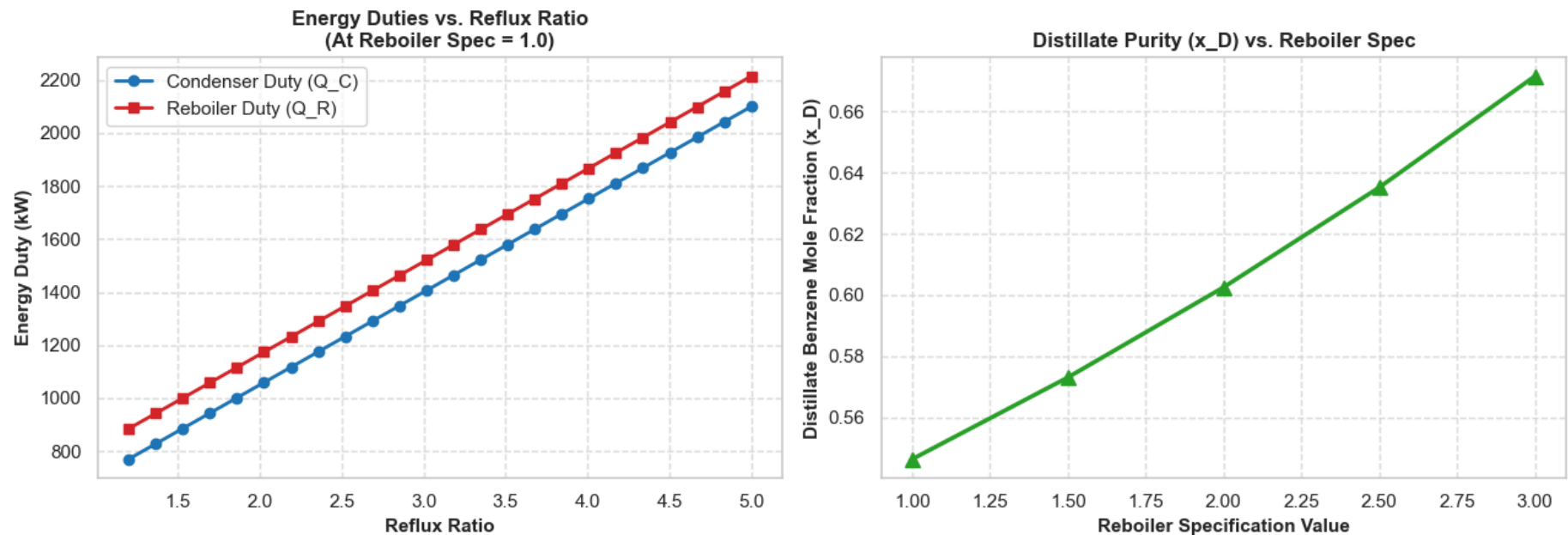
axes[1].plot(df_sub_reboiler['Reboiler_Spec'], df_sub_reboiler['x_D'], '^-', color='#2ca02c', linewidth=2.5, markersize=8)
axes[1].set_title('Distillate Purity (x_D) vs. Reboiler Spec', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Reboiler Specification Value', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Distillate Benzene Mole Fraction (x_D)', fontsize=11, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.suptitle('Physical Sensitivity Trend Lines', fontsize=14, fontweight='bold')
plt.tight_layout()

plt.savefig('Physical_Sensitivity_Trends.png', dpi=300)
plt.show()

```

Physical Sensitivity Trend Lines



MODEL ERROR METRIC(MAE) COMPARISON GRAPH

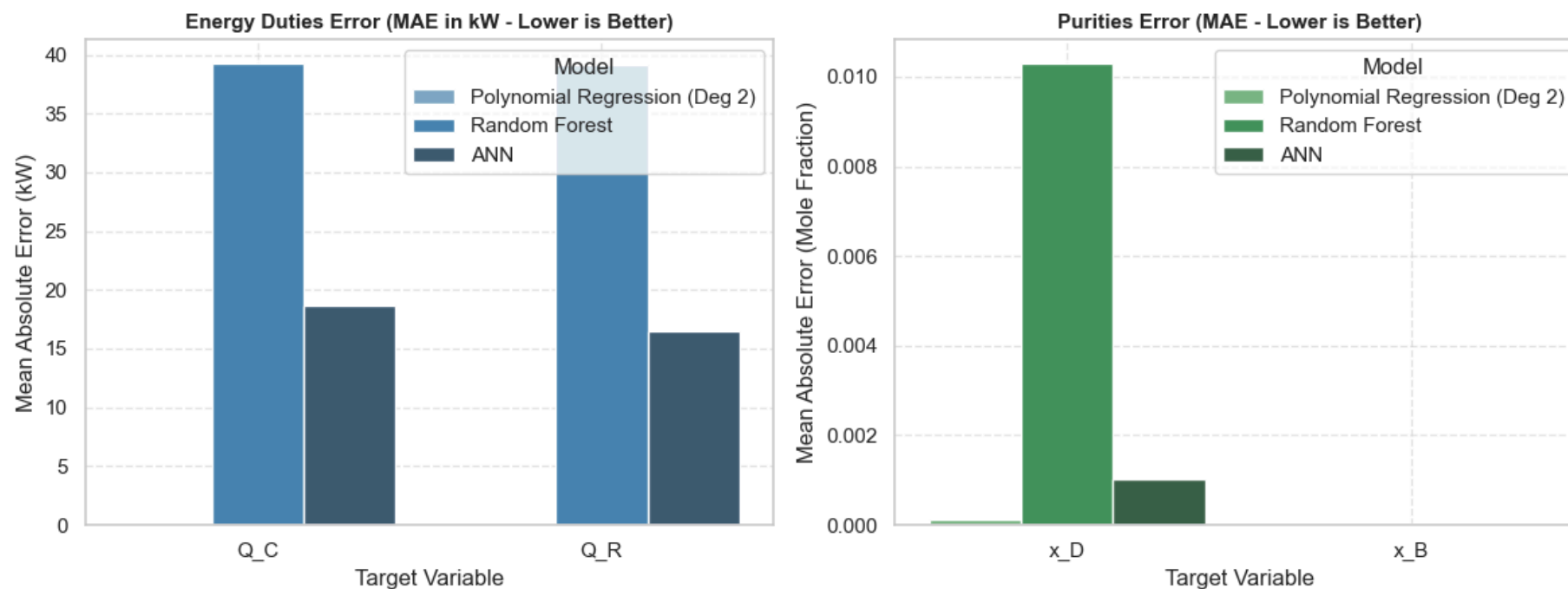
```
In [37]: sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sns.barplot(
    data=eval_df[eval_df['Target'].isin(['Q_C', 'Q_R'])],
    x='Target',
    y='MAE',
    hue='Model',
    ax=axes[0],
    palette='Blues_d'
)
axes[0].set_title('Energy Duties Error (MAE in kW - Lower is Better)', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Mean Absolute Error (kW)')
axes[0].set_xlabel('Target Variable')
axes[0].grid(True, linestyle='--', alpha=0.5)

sns.barplot(
    data=eval_df[eval_df['Target'].isin(['x_D', 'x_B'])],
    x='Target',
    y='MAE',
    hue='Model',
    ax=axes[1],
    palette='Greens_d'
)
axes[1].set_title('Purities Error (MAE - Lower is Better)', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Mean Absolute Error (Mole Fraction)')
axes[1].set_xlabel('Target Variable')
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.suptitle('Model Error Metric Comparison (MAE) Graph', fontsize=14, fontweight='bold')
plt.tight_layout()

plt.savefig('MAE_Comparison_Bar_Chart.png', dpi=300)
plt.show()
```

Model Error Metric Comparison (MAE) Graph



SENSITIVITY ANALYSIS TREND GRAPH

```
In [31]: sns.set_theme(style="whitegrid")

# Create figure with 2 subplots for Sensitivity Analysis Trends
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# SUBPLOT 1: DUAL-AXIS SENSITIVITY (Q_R and x_D vs Reflux Ratio)
# Isolate Reflux Ratio variation by fixing Reboiler Spec at its median value (2.0)
median_reboiler = df['Reboiler_Spec'].median()
df_sens = df[df['Reboiler_Spec'] == median_reboiler].sort_values('Reflux_Ratio')

# Primary Axis: Reboiler Duty (Q_R) in kW
color_qr = '#d62728'
axes[0].plot(
    df_sens['Reflux_Ratio'],
```

```

    df_sens['Q_R'],
    'o-',
    color=color_qr,
    linewidth=2.5,
    markersize=6,
    label='Reboiler Duty (Q_R)'
)
axes[0].set_xlabel('Reflux Ratio', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Reboiler Duty Q_R (kW)', color=color_qr, fontsize=11, fontweight='bold')
axes[0].tick_params(axis='y', labelcolor=color_qr)
axes[0].grid(True, linestyle='--', alpha=0.5)

# Secondary Axis: Distillate Benzene Purity (x_D)
ax_xD = axes[0].twinx()
color_xd = '#1f77b4'
ax_xD.plot(
    df_sens['Reflux_Ratio'],
    df_sens['x_D'],
    's--',
    color=color_xd,
    linewidth=2,
    markersize=6,
    label='Distillate Purity (x_D)'
)
ax_xD.set_ylabel('Distillate Benzene Purity (x_D)', color=color_xd, fontsize=11, fontweight='bold')
ax_xD.tick_params(axis='y', labelcolor=color_xd)
ax_xD.grid(False)

axes[0].set_title(f'Sensitivity Analysis: Impact of Reflux Ratio\n(At Fixed Reboiler Spec = {median_reboiler})', fontsize=12,

# SUBPLOT 2: MULTI-LINE SENSITIVITY OF Q_R ACROSS ALL REBOILER SPECS
sns.lineplot(
    data=df,
    x='Reflux_Ratio',
    y='Q_R',
    hue='Reboiler_Spec',
    palette='viridis',
    marker='o',
    ax=axes[1],
    linewidth=2
)

```

```

axes[1].set_title('Reboiler Duty (Q_R) vs. Reflux Ratio across Reboiler Specs', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Reflux Ratio', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Reboiler Duty Q_R (kW)', fontsize=11, fontweight='bold')
axes[1].grid(True, linestyle='--', alpha=0.5)
axes[1].legend(title='Reboiler Spec', loc='upper left')

# Super Title & Layout Adjustment
plt.suptitle('Sensitivity Analysis Trend Plots', fontsize=14, fontweight='bold')
plt.tight_layout()

# Save image for final report submission
plt.savefig('Sensitivity_Analysis_Trends.png', dpi=300)
plt.show()

```

Sensitivity Analysis Trend Plots

